

Cost Estimation

Project Cost Management

- **Project cost management** includes the processes required to ensure that a project team completes **a project** within an **approved budget**.
- The project manager's job is to satisfy project stakeholders while continuously striving to reduce and control costs.

Key Processes in Project Cost Management

- *Planning cost management*
- *Estimating costs and determining the budget*
- *Controlling costs*

Project/Software Cost- Parameters

- Size: Usually measured in **LOC** or the number of **Function Points** required to realize the desired capabilities.
 - Function Points – a better metric earlier in the project
 - LOC (SLOC, KLOC...) a better metric later in the project
- Project/Process Complexity: – Complex projects with intricate requirements, integration challenges, or advanced functionalities may require more resources and budget.

Process

Project/Software Cost- Parameters

- Personnel – capabilities of the personnel in general and in the application domain in particular
- All human resources involved in the project, from technical staff to project managers. Skilled labor is often the largest cost component, especially in technical or labor-intensive projects.
 - There is much specialization nowadays. Some are expensive.
 - Emphasize ‘team’ and team responsibilities...Ability to work in a team;
 - Several newer lightweight methodologies are built around a team or tiny group of individuals...

Project/Software Cost- Parameters

- Environment – The tools/techniques / automated procedures used to support the development effort.
 - Integrated tools; automated tools for modelling, testing, configuration, managing change, defect tracking, etc...
- Required Quality of the Project – the functionality provided; performance, reliability, maintainability, scalability, portability, user interface utility; usability...

Project/Software Cost- Parameters

Effort = (personnel)(environment)(quality)(size)(process)

Cost of a Project

- The cost in a project is due to:
 - due to the requirements for software, hardware, and human resources
 - the cost of software development is due to the human resources needed
 - most cost estimates are measured in ***person-months (PM)***
- The cost of the project depends on the **nature** and **characteristics** of the project,
- At any point, the estimate's accuracy will depend on the amount of **reliable information** we have about the final product.

Software/Project Cost Estimation

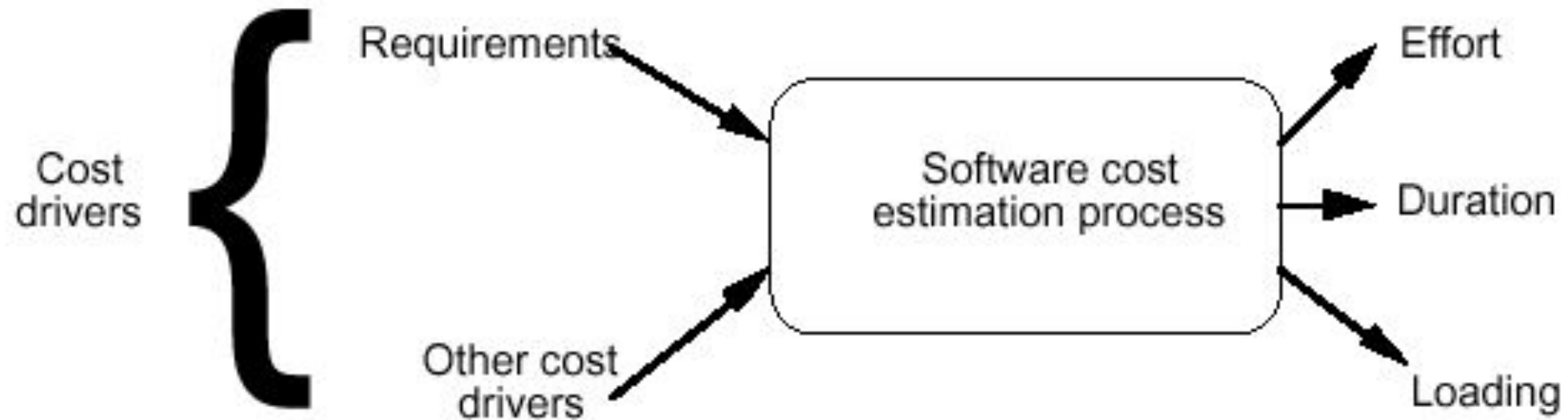


Figure 1. Classical view of software estimation process.

Introduction to COCOMO models

- The **COstructive COst Model** (COCOMO) is the most widely used software estimation model
- The COCOMO model predicts the **effort, resource,** and **duration** of a project based on inputs relating to the size of the resulting systems and a number of "**cost drives**" that affect productivity.

Effort

- Effort Equation

- $PM = C * (KDSI)^n$ (person-months)

- where **PM** = number of person-month (person-152 working hours),
 - **C** = a constant,
 - **KDSI** = thousands of "delivered source instructions" (DSI) and
 - **n** = a constant.

Productivity

- Productivity equation
 - **(DSI) / (PM)**
 - where **PM** = number of person-month (=152 working hours),
 - **DSI** = "delivered source instructions"

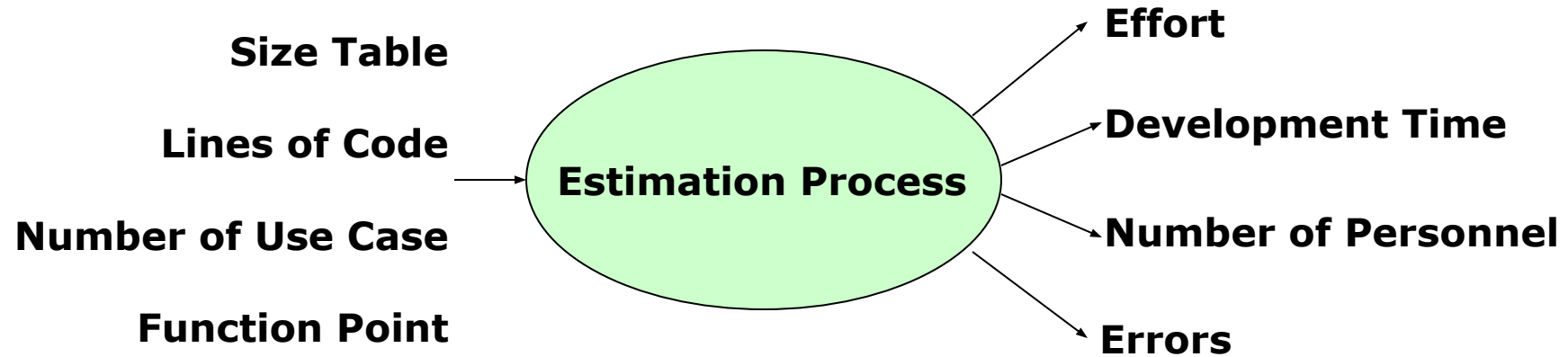
Schedule

- Schedule equation
 - **$TDEV = C * (PM)^n$** (months)
 - where TDEV = number of months estimated for software development.

Average Staffing

- Average Staffing Equation
 - $(PM) / (TDEV) \quad (FSP)$
 - where FSP means Full-time-equivalent Software Personnel.

Cost Estimation Process



Types of Software Projects Mode

- The most important factor contributing to a project's duration and cost is the Development Mode
 - **Organic Mode:** The project is developed in a familiar, stable environment, and the product is similar to previously developed products. The product is relatively small and requires little innovation.
 - **Semidetached Mode:** The project's characteristics are intermediate between Organic and Embedded. development consisting of a mix of experienced and inexperienced staff.
 - **Embedded Mode:** The project is characterized by tight, inflexible constraints and interface requirements. An embedded mode project will require a great deal of innovation.

COCOMO I

- COCOMO 1 is primarily based on the number of lines of code (LOC) as the primary measure of software size.
- It provides a good rough estimation
- COCOMO has three different **models** (each one increasing with detail and accuracy):
 - **Basic**, applied early in a project. Use for quick and rough cost calculations for the software
 - **Intermediate**, applied after requirements are specified.
 - **Advanced**, applied after the design is complete
- COCOMO has three different **modes**:
 - **Organic** – “relatively small software teams develop software in a highly familiar, in-house environment”
 - **Embedded** – operate within tight constraints, product is strongly tied to “complex of hardware, software, regulations, and operational procedures”
 - **Semi-detached** – intermediate stage somewhere between organic and embedded. Usually up to 300 KDSI

BASIS COCOMO

- COCOMO uses two equations to calculate effort in Person-Months (PM) and the number of months estimated for the project (TDEV)
- PM (or Effort) is based on the number of thousand lines of delivered instructions/source (KDSI)
- $PM = a(KDSI)^b * \text{EAF (EAF for intermediate model)}$
- $TDEV = c(PM)^d$
- EAF is the **Effort Adjustment Factor** derived from the Cost Drivers, EAF for the basic model is 1
- The values for a, b, c, and d differ depending on which mode you are using

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

COCOMO 1

- A simple example:

Project is a flight control system (mission critical) with 310,000 DSI in embedded mode

- **Reliability must be very high (RELY=1.40).** So we can calculate:
- **Effort** = $1.40 * 3.6 * (310)^{1.20} = 5093 \text{ MM}$
- **Schedule** = $2.5 * (5093)^{0.32} = 38.4 \text{ months}$
- **Average Staffing** = $5093 \text{ MM} / 38.4 \text{ months} = 133 \text{ FSP}$

Intermediate COCOMO

- The **intermediate model** is an extension of the basic model and includes a set of cost drivers to calculate the estimates with better accuracy. The effort factor includes the **effort adjustment factor (EAF)** that is calculated with the cost drivers.
- $PM \text{ (or Effort)} = a(KDSI)^b * EAF$
- $TDEV = c(PM)^d$

Intermediate COCOMO-Cost drivers

Product Attributes

Product Attributes	Very Low	Low	Nominal	High	Very High	Extra High
RELY	0.75	0.88	1.00	1.15	1.40	...
DATA	...	0.94	1.00	1.08	1.16	...
CPLX	0.70	0.85	1.00	1.15	1.30	1.65

RELY: Reliability

DATA: Data size

CPLX = Complexity

Intermediate COCOMO-Cost drivers

- **Hardware attributes**

Hardware Attributes	Very Low	Low	Nominal	High	Very High	Extra High
TIME	...	...	1.00	1.11	1.30	1.66
STOR	...	...	1.00	1.06	1.21	1.56
VIRT	...	0.87	1.00	1.15	1.30	...
TURN	...	0.87	1.00	1.07	1.15	...

Time= Run time performance constraints
TURN= Required turnaround time

STOR: Memory constraints

VIRT= Volatility of the virtual machine

Intermediate COCOMO-Cost drivers

Personal attributes

- Analyst capabilities
- Software engineering capabilities
- Applications experience
- Virtual machine experience
- Programming language experience

Personal attributes	Very Low	Low	Nominal	High	Very High	Extra High
ACAP	1.46	1.19	1.00	0.86	0.71	...
AXEP	1.29	1.13	1.00	0.91	0.82	...
PCAP	1.42	1.17	1.00	0.86	0.70	...
VEXP	1.21	1.10	1.00	0.90	...	...
LEXP	1.14	1.07	1.00	0.95	...	...

Example

- Suppose a project was estimated to be made in 400 kLOC. let's calculate its effort, time, and the number of people required while considering the project is of organic type and has a nominal complexity. The developer has a high virtual machine experience.
- The value of the nominal complexity of a project is 1.00, and the high virtual experience of the developer is 0.90, according to the tables mentioned above:

$$EAF=1.00*0.90=0.9$$

$$Effort(E)=2.4(400\text{ kLOC})1.05*0.9=1554.37\text{ person-months}$$

$$Time(T)=2.5(1554.37)0.38=40.80\text{ months}$$

$$People\ required=1554.37/40.80=38.09\text{ persons}$$

Detailed Model of COCOMO

- Detailed COCOMO incorporates all characteristics of the intermediate version with an assessment of the cost driver's impact on each step of the software engineering process. The Six phases of detailed COCOMO are:
 - Planning and requirements
 - System design
 - Detailed design
 - Module code and test
 - Integration and test
 - Cost Constructive model

Function Point Analysis (FPA)

- A method used for software cost estimation based on the functionality provided by the software. It quantifies the software's functionality by measuring the number of function points, which represent the inputs, outputs, inquiries, internal logical files, and external interfaces.

Function Point Analysis (FPA)..

- **STEP 1: Identify Function Types**
- Function points are computed by first calculating an ***unadjusted function point count (UFC)***. Counts are made for the following categories
 - *External inputs (User Input)* - Represent user inputs that affect the internal processing of the software and result in data being maintained or altered by the system. Examples include **forms, data entry screens, or file imports**.
 - *External outputs* – These are outputs the system provides to the user, which involve processing. Such as **reports and messages**.

Function Point Analysis (FPA)..

- *External inquiries* – These are interactive inputs that retrieve data from the system without altering any data. External Inquiries capture user requests for information from the system and the system's response in providing that information. **Examples are search forms or queries.**
- *External Interface files* – These are external data files accessed by the system but not maintained by it. E.g., machine-readable interfaces to other systems
- *Internal files* – The internal data maintained by the system, like database tables or files created and used within the application. Logical master files in the system

Function Point Analysis (FPA)..

- **STEP 2:** Multiply each number by a weight factor, according to the complexity (**simple**, **average** or **complex**) of the parameter, associated with that number and find the Unadjusted Function Point (**UFP**). The value is given by a table:

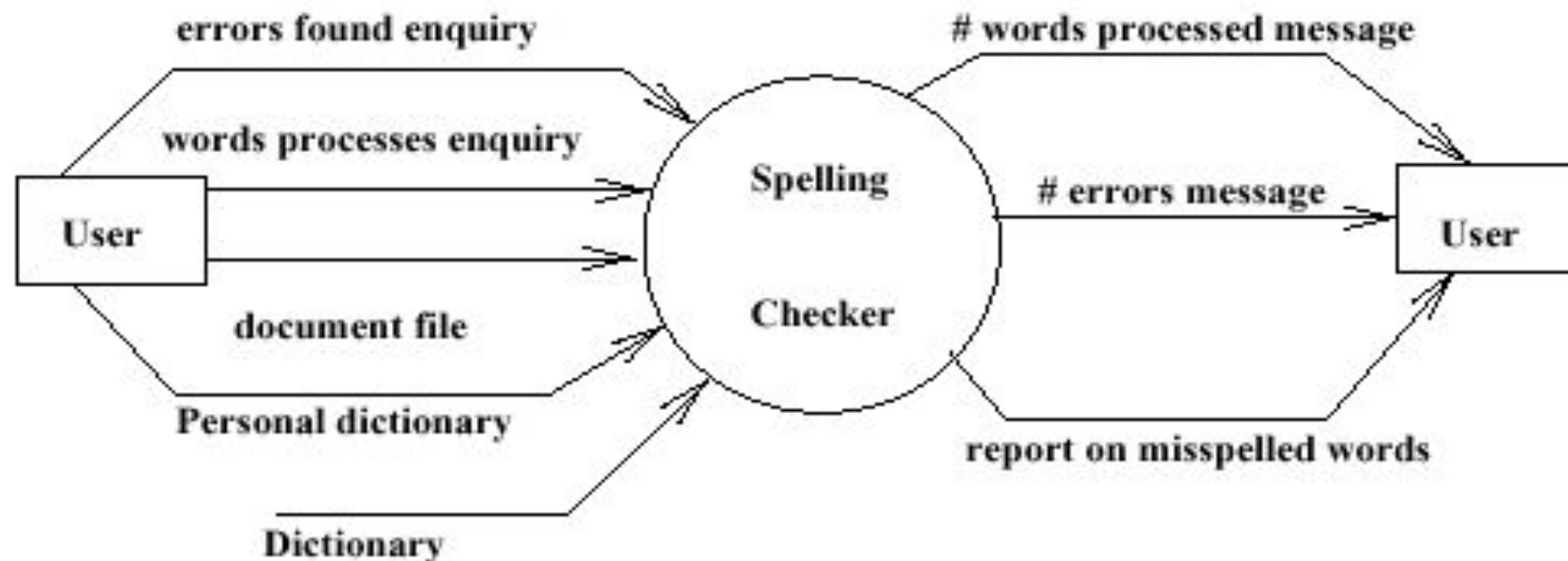
Parameter	simple	average	complex
users inputs	3	4	6
users outputs	4	5	7
users requests	3	4	6
files	7	10	15
external interfaces	5	7	10

Function Point Analysis (FPA)..

- **STEP 3**: Find **DI** (Degree of Influence) using the **TCF** table (*a separate table*)
- **STEP 4**: Final **TCF** (Technical Complexity Factor) given by the formula
 - **$TCF = 0.65 + 0.01 * DI$**
- **STEP 5**: Function Points are given by the formula
 - **$FP = UFP * TCF$**

Example

The Spell-Checker accepts as input a document file and an optional personal dictionary file. The checker lists all words not contained in either of these files. The user can query the number of words processed and the number of spelling errors found at any stage during processing.



Example (.)

- 2 users inputs: document file name, personal dictionary name (average)
- 3 users outputs: fault report, word count, misspelled error count (average)
- 2 users requests: #treated words?, #found errors? (average)
- 1 internal file: dictionary (average)
- 2 external files: document file, personal dictionary (av).

$$UFP = 4 \times 2 + 5 \times 3 + 4 \times 2 + 10 \times 1 + 7 \times 2 = 55$$

Example (..)

Technical Complexity Factors for Function Points: (0-5)

- 1. Data Communication 3
- 2. Distributed Data Processing 0
- 3. Performance Criteria 4
- 4. Heavily Utilized Hardware 0
- 5. High Transaction Rates 3
- 6. Online Data Entry 3
- 7. Online Updating 3
- 8. End-user Efficiency 3
- 9. Complex Computations 0
- 10. Reusability 2
- 11. Ease of Installation 3
- 12. Ease of Operation 4
- 13. Portability 0
- 14. Maintainability 3

- DI =30 (Degree of Influence)

https://www.tutorialspoint.com/software_quality_management/software_quality_management_albrechts_function_point_method.htm

Example (...)

- Function Points

- $FP = UFP * (0.65 + 0.01 * DI) = 55 * (0.65 + 0.01 * 30) = 52.25$

- That means the is **FP=52.25**

Relation between LOC and FP

- Relationship:

- ***LOC = Language Factor * FP***

- where

- **LOC** (Lines of Code)
 - **FP** (Function Points)

Relation between LOC and FPs

Language	LOC/FP
assembly	320
C	128
Cobol	105
Fortan	105
Pascal	90
Ada	70
OO languages	30
4GL languages	20

Relation between LOC and FP(.)

Assuming LOC's per FP for:

Java = 53,

C++ = 64

$$\mathbf{aKLOC = FP * LOC_per_FP / 1000}$$

It means for the SpellChekcer Example: (Java)

$$\mathbf{LOC=52.25*53=2769.25 \text{ LOC or } 2.76 \text{ KLOC}}$$

Effort Computation

- The **Basic COCOMO model** computes effort as a function of program size. The Basic COCOMO equation is:
 - $E = aKLOC^b$
- Effort for three modes of Basic COCOMO.

Mode	a	b
<i>Organic</i>	2.4	1.05
<i>Semi-detached</i>	3.0	1.12
<i>Embedded</i>	3.6	1.20

Example

Mode	Effort Formula
Organic	$E = 2.4 * (S^{1.05})$
Semidetached	$E = 3.0 * (S^{1.12})$
Embedded	$E = 3.6 * (S^{1.20})$

Size = 200 KLOC

Effort = $a * \text{Size}^b$

Organic — $E = 2.4 * (200^{1.05}) = 626$ staff-months

Semidetached — $E = 3.0 * (200^{1.12}) = 1133$ staff-months

Embedded — $E = 3.6 * (200^{1.20}) = 2077$ staff-months

Effort Computation

- The **intermediate COCOMO model** computes effort as a function of program size and a set of cost drivers. The Intermediate COCOMO equation is:
 - $E = aKLOC^b * EAF$
- Effort for three modes of intermediate COCOMO.

Mode	a	b
<i>Organic</i>	3.2	1.05
<i>Semi-detached</i>	3.0	1.12
<i>Embedded</i>	2.8	1.20

Effort computation(.)

- **Effort Adjustment Factor**

Cost Driver	Very Low	Low	Nominal	High	Very High	Extra High
Required Reliability	.75	.88	1.00	1.15	1.40	1.40
Database Size	.94	.94	1.00	1.08	1.16	1.16
Product Complexity	.70	.85	1.00	1.15	1.30	1.65
Execution Time Constraint	1.00	1.00	1.00	1.11	1.30	1.66
Main Storage Constraint	1.00	1.00	1.00	1.06	1.21	1.56
Virtual Machine Volatility	.87	.87	1.00	1.15	1.30	1.30
Comp Turn Around Time	.87	.87	1.00	1.07	1.15	1.15
Analyst Capability	1.46	1.19	1.00	.86	.71	.71
Application Experience	1.29	1.13	1.00	.91	.82	.82
Programmers Capability	1.42	1.17	1.00	.86	.70	.70
Virtual machine Experience	1.21	1.10	1.00	.90	.90	.90
Language Experience	1.14	1.07	1.00	.95	.95	.95
Modern Prog Practices	1.24	1.10	1.00	.91	.82	.82
SW Tools	1.24	1.10	1.00	.91	.83	.83
Required Dev Schedule	1.23	1.08	1.00	1.04	1.10	1,10